

FATEC POMPEIA

Introdução à Computação em Nuvem – ISW055 – 161_SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual – 2026.2

Lucas Tetsuya Hiratuca

Prof. Allan Lincoln Rodrigues Siriani

Pompeia, 07/10/2026

Sumário

Introdução	3
Metodologia	3
Quadro de entregas	3
Atividades realizadas	4
Atividade 1 – Agenda telefônica em Flask	4
Atividade 2 – Catálogo de filmes – Tom Hanks	6
Atividade 3 – Desacoplando o login – microsserviço de autenticação	7
Atividade 4 – Controle de acesso por papel – RBAC	8
Atividade 5 – Logs e auditoria	9
Atividade 6 – Upload e perfil de usuário	11
Considerações finais	13
Declaração de autoria	13

Introdução

A disciplina **ISW055 – Introdução à Computação em Nuvem**, ministrada pelo Prof. Allan Lincoln Rodrigues Siriani na FATEC Pompeia no semestre letivo 2026.2, tem como propósito central proporcionar aos alunos a vivência prática e conceitual na construção, containerização e orquestração de aplicações modernas para ambientes em nuvem.

O projeto condutor ao longo deste primeiro bimestre foi o desenvolvimento e a evolução do **Catálogo de Filmes de Tom Hanks**. Partindo de uma arquitetura inicial monolítica, a solução progrediu de forma incremental para uma infraestrutura desacoplada em microsserviços containerizados com Docker. Foram incorporados requisitos reais de engenharia de software em nuvem: autenticação segura via JSON Web Tokens (JWT), controle de acesso baseado em papéis (RBAC), auditoria assíncrona de eventos com Redis Streams e persistência de mídia em object storage compatível com S3 (Garage S3).

Este relatório bimestral documenta minuciosamente as seis atividades propostas e entregues, detalhando as decisões de arquitetura adotadas, as dificuldades técnicas superadas, as evidências de entrega com carimbo temporal no Git e os resultados operacionais comprovados do sistema.

Metodologia

Todas as atividades foram desenvolvidas com controle de versão via Git em repositórios públicos no GitHub. As datas e os horários de entrega foram rigorosamente extraídos do histórico de commits (`git log`) e conferidos diretamente na interface web do GitHub, garantindo rastreabilidade e imutabilidade das comprovações.

O ambiente de execução e teste foi estruturado com **Docker Desktop** e **Docker Compose**, padronizando os contêineres entre os ambientes de desenvolvimento e produção. O deploy final em nuvem é gerenciado via **Portainer** no servidor disponibilizado pelo orientador (`lucas-hiratuka-isw055.lapps.studio`), exposto por túnel seguro Cloudflare. As imagens oficiais foram publicadas no Docker Hub no namespace `lucashiratuka/`.

Os testes de conformidade e segurança foram realizados com suítes automatizadas locais e no contêiner, inspecionando o cumprimento das diretrizes do OWASP Top 10:2021.

Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em MariaDB.	07/08/2026	07/08/2026 20:03	entregue
2	Catálogo de filmes – Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	20/08/2026 20:54	entregue
3	Desacoplando o login – microsserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	27/08/2026 13:00	entregue
4	Controle de acesso por papel – RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	01/09/2026 00:48	entregue
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	08/09/2026 12:57	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no Garage S3; só a referência fica no banco relacional.	02/10/2026	26/09/2026 14:43	entregue

Atividades realizadas

Atividade 1 – Agenda telefônica em Flask

Campo	Valor
Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em banco de dados MariaDB na nuvem.
Data planejada	07/08/2026
Data realizada	07/08/2026 20:03
Situação	entregue
Evidência	GitHub – commits 28230dc e 738ce73 + README com menção ao professor + print do sistema
Link	#link

O que foi feito. Durante a aula inicial de nivelamento, foi desenvolvida uma aplicação web em Python utilizando o framework Flask e o motor de templates Jinja2. A aplicação implementa um CRUD de contatos telefônicos conectado a uma instância remota do MariaDB hospedada no IP 35.226.64.52.

Como decisão técnica, a camada de acesso a dados foi isolada utilizando a biblioteca PyMySQL, configurando codificação utf8mb4 e tratamento de erros de conexão. Posteriormente, foi aplicado saneamento de credenciais via variáveis de ambiente com arquivo .gitignore (commit fa5f58d).

cole o print aqui

troque arquivo: none por arquivo: "prints/nome.png"

Listagem 1: Atividade 1 – evidência da entrega (link, data e hora visíveis)

cole o print aqui

troque arquivo: none por arquivo: "prints/nome.png"

Listagem 2: Atividade 1 – resultado (o sistema/mapa/artigo funcionando)

Dificuldades e como foram resolvidas. A principal dificuldade inicial consistiu na latência e na autenticação segura contra a base de dados remota do laboratório. O problema foi sanado com o correto ajuste da porta MySQL e a parametrização das variáveis no ambiente local.

Atividade 2 – Catálogo de filmes – Tom Hanks

Campo	Valor
Descrição	Consumo da API TMDb, persistência em MariaDB e segregação por usuário com sistema de favoritos.
Data planejada	20/08/2026
Data realizada	20/08/2026 20:54
Situação	entregue
Evidência	GitHub – commit 9a57391 + README com menção ao professor + print do catálogo
Link	#link

O que foi feito. Criação da aplicação principal do projeto utilizando Node.js, Express e EJS. A aplicação consome a API pública do The Movie Database (TMDb), filtrando e exibindo a filmografia completa do ator Tom Hanks com posters, títulos originais, sinopses e notas de avaliação.

Foi construído o sistema de autenticação local com `express-session` e senhas criptografadas com `bcryptjs`. Adicionalmente, foi implementada a funcionalidade de filmes favoritos vinculada ao ID do usuário autenticado no MariaDB, segregando a lista de favoritos por sessão ativa.

cole o print aqui

troque arquivo: none por arquivo: "prints/nome.png"

Listagem 3: Atividade 2 – evidência da entrega (link, data e hora visíveis)

cole o print aqui

troque arquivo: none por arquivo: "prints/nome.png"

Listagem 4: Atividade 2 – resultado (o sistema/mapa/artigo funcionando)

Dificuldades e como foram resolvidas. A API do TMDb utiliza paginação de resultados. Para garantir que toda a filmografia do ator fosse renderizada sem cortes, foi implementada uma rotina assíncrona que itera sobre as páginas da API e consolida os dados em memória antes da renderização EJS.

Atividade 3 – Desacoplando o login – microsserviço de autenticação

Campo	Valor
Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	27/08/2026 13:00
Situação	entregue
Evidência	GitHub – commit 320028a + docker-compose.yml + print do login funcionando
Link	#link

O que foi feito. A arquitetura do sistema foi refatorada sob o padrão Backend-for-Frontend (BFF). O serviço de catálogo (catalogo-service) foi mantido como único ponto de entrada público (porta 8217), enquanto toda a lógica de identidade, login, cadastro de novos usuários e redefinição de senha foi isolada no novo auth-service (porta interna 4000).

O auth-service não expõe portas ao host, comunicando-se exclusivamente pela rede interna Docker app_network. Ele emite tokens JWT criptograficamente assinados contendo a identidade do usuário. O fluxo de recuperação de senha foi integrado ao serviço SMTP do Mailtrap para despacho de e-mails com tokens descartáveis com expiração de 30 minutos.

cole o print aqui

troque arquivo: none por arquivo: "prints/nome.png"

Listagem 5: Atividade 3 – evidência da entrega (link, data e hora visíveis)

cole o print aqui

troque arquivo: none por arquivo: "prints/nome.png"

Listagem 6: Atividade 3 – resultado (o sistema/mapa/artigo funcionando)

Dificuldades e como foram resolvidas. Ajustar a comunicação segura entre contêineres na mesma rede virtual do Docker Compose sem depender de IPs estáticos. A resolução foi alcançada utilizando os nomes dos serviços como hostnames DNS internos do Docker (`http://auth-service:4000`).

Atividade 4 – Controle de acesso por papel – RBAC

Campo	Valor
Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	01/09/2026 00:48
Situação	entregue
Evidência	GitHub – commits 6636f4d e a8e31ef + print do 403 e da ação de admin
Link	#link

O que foi feito. Implementou-se o modelo de autorização Role-Based Access Control (RBAC) adotando o Padrão B (claims embutidas diretamente no payload do JWT). O campo role (usuario ou admin) trafega assinado no token, permitindo verificação instantânea no catalogo-service sem overhead de consultas repetidas ao banco de dados.

Foram criados os middlewares requireLogin e requireAdmin. Usuários autenticados com perfil comum têm acesso restrito ao catálogo, favoritos e comentários próprios. Tentativas de acessar rotas de moderação ou painel de gestão de usuários (/admin/usuarios) são barradas com status HTTP 403 Forbidden diretamente na camada do servidor.

cole o print aqui

troque arquivo: none por arquivo: "prints/nome.png"

Listagem 7: Atividade 4 – evidência da entrega (link, data e hora visíveis)

ID	NOME	E-MAIL	PAPEL ATUAL	CADASTRO	AÇÃO
5	sou eu	lucasussy69@gmail.com	Usuário	01/09/2026	Promover a Admin
4	jjj	tetsuyachan2@gmail.com	Usuário	31/08/2026	Promover a Admin
3	TetsuyaChan	tetsuyachan@gmail.com	Usuário	21/08/2026	Promover a Admin
2	Philia093	flawlessego@email	Admin	20/08/2026	Rebaixar a Usuário
1	n sei	a@a.com	Admin	20/08/2026	Você

Figura 1: Atividade 4 – resultado (painel de administração de usuários e RBAC em funcionamento)

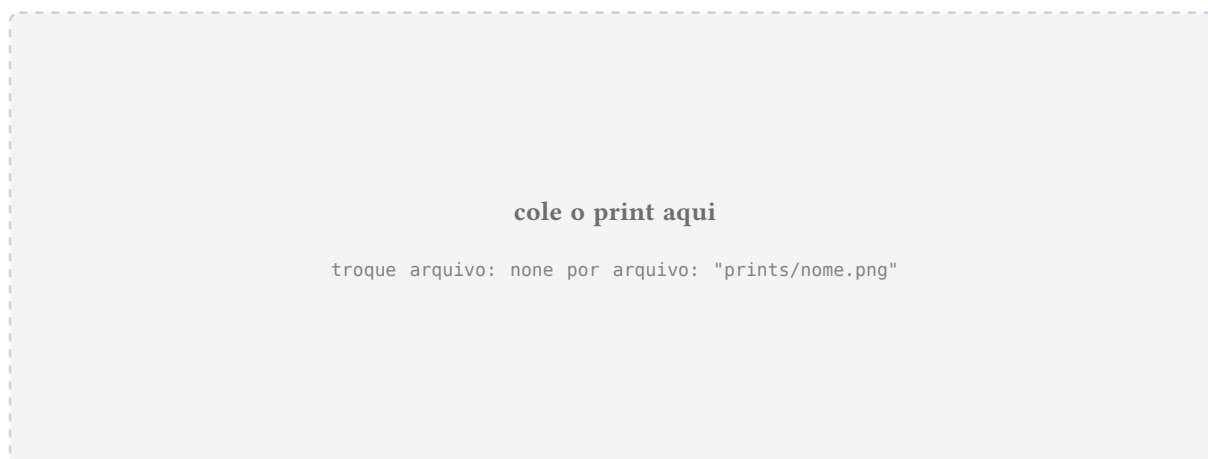
Dificuldades e como foram resolvidas. Garantir que tipos de dados inteiros para userId não causassem falhas na comparação estrita (===) no JavaScript. Foi adotado parsing consistente com parseInt() em todos os middlewares e controladores de permissão.

Atividade 5 – Logs e auditoria

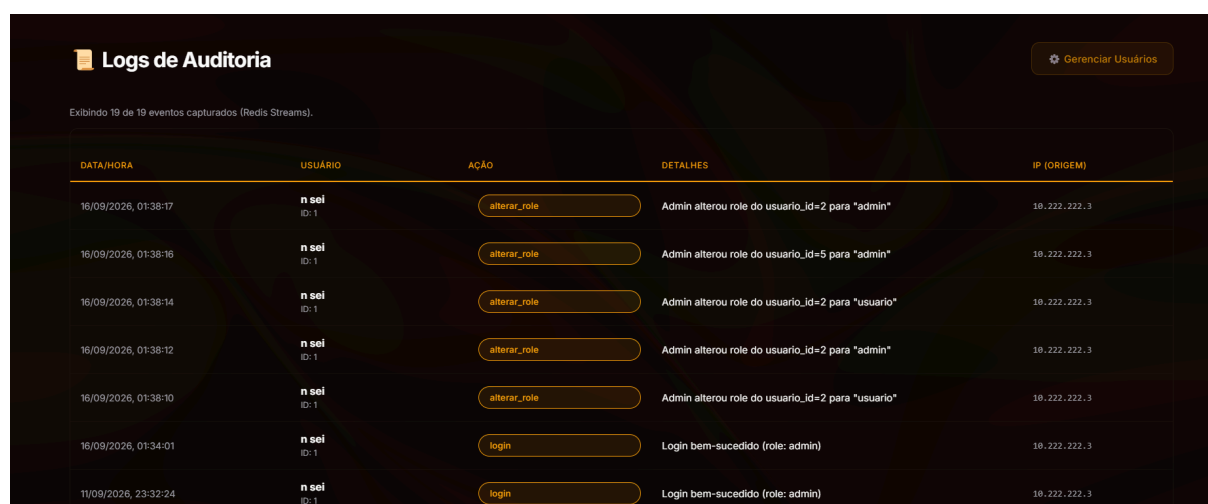
Campo	Valor
Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	08/09/2026 12:57
Situação	entregue
Evidência	GitHub – commits e6c6682 e 8bc9faa + print da consulta de logs pelo admin
Link	#link

O que foi feito. Foi introduzido um terceiro microsserviço dedicado, o `log-service` (porta 5000), acoplado a uma instância do Redis 7. O serviço implementa ingestão assíncrona de eventos via HTTP POST, persistindo as entradas de auditoria na estrutura de alta performance Redis Streams.

O `catalogo-service` consome o utilitário de auditoria registrando eventos críticos: logins com sucesso, encerramentos de sessão, falhas de autenticação, exclusão de comentários e edições de perfil. A interface `/admin/logs` foi criada com restrição exclusiva a administradores para visualização e filtragem cronológica reversa dos registros em tempo real.



Listagem 8: Atividade 5 – evidência da entrega (link, data e hora visíveis)



DATA/HORA	USUÁRIO	AÇÃO	DETALHES	IP (ORIGEM)
16/09/2026, 01:38:17	n sei ID: 1	alterar_rola	Admin alterou role do usuario_id=2 para "admin"	10.222.222.3
16/09/2026, 01:38:16	n sei ID: 1	alterar_rola	Admin alterou role do usuario_id=5 para "admin"	10.222.222.3
16/09/2026, 01:38:14	n sei ID: 1	alterar_rola	Admin alterou role do usuario_id=2 para "usuario"	10.222.222.3
16/09/2026, 01:38:12	n sei ID: 1	alterar_rola	Admin alterou role do usuario_id=2 para "admin"	10.222.222.3
16/09/2026, 01:38:10	n sei ID: 1	alterar_rola	Admin alterou role do usuario_id=2 para "usuario"	10.222.222.3
16/09/2026, 01:34:01	n sei ID: 1	login	Login bem-sucedido (role: admin)	10.222.222.3
11/09/2026, 23:32:24	n sei ID: 1	login	Login bem-sucedido (role: admin)	10.222.222.3

Figura 2: Atividade 5 – resultado (tela de consulta de logs de auditoria via Redis Streams)

Dificuldades e como foram resolvidas. Garantir a sobrevivência dos logs a reinicializações de contêineres. Foi configurado um volume nomeado persistente para o contêiner do Redis no `docker-compose.yml`, além de

timeout não-bloqueante no envio de logs para evitar impacto na experiência do usuário final caso o Redis ficasse temporariamente indisponível.

Atividade 6 – Upload e perfil de usuário

Campo	Valor
Descrição	Página de perfil com avatar no Garage S3; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	26/09/2026 14:43
Situação	entregue
Evidência	GitHub – commits 4219f05, 27856a8 e 8ca3fb6 + print do perfil com foto + SECURITY_AUDIT.md
Link	#link

O que foi feito. Construção da página de perfil com suporte a upload de foto de avatar persistida em object storage. Foi integrado o **Garage S3** (`dxflrs/garage:v1.0.1`), um engine leve e distribuído escrito em Rust compatível com AWS S3 API (Signature V4). A tabela perfis no MariaDB armazena apenas a biografia e a chave textual do objeto (`foto_key`), desacoplando o binário do banco relacional.

A entrega da imagem foi arquitetada via **streaming seguro pelo backend** (`/perfil/:userId/foto`): o `catalogo-service` obtém o stream do Garage S3 via rede interna e o devolve ao navegador com cabeçalhos HTTP adequados (`Content-Type` validado e `Cache-Control`). O upload exige validação de MIME type (JPEG/PNG/WebP/GIF), limite de 5MB e geração de nomes via `randomUUID()`. Foi realizada ainda auditoria OWASP Top 10 com 100% de aprovação e remediação do CWE-640 (Password Reset Poisoning).

cole o print aqui

```
troque arquivo: none por arquivo: "prints/nome.png"
```

Listagem 9: Atividade 6 – evidência da entrega (link, data e hora visíveis)

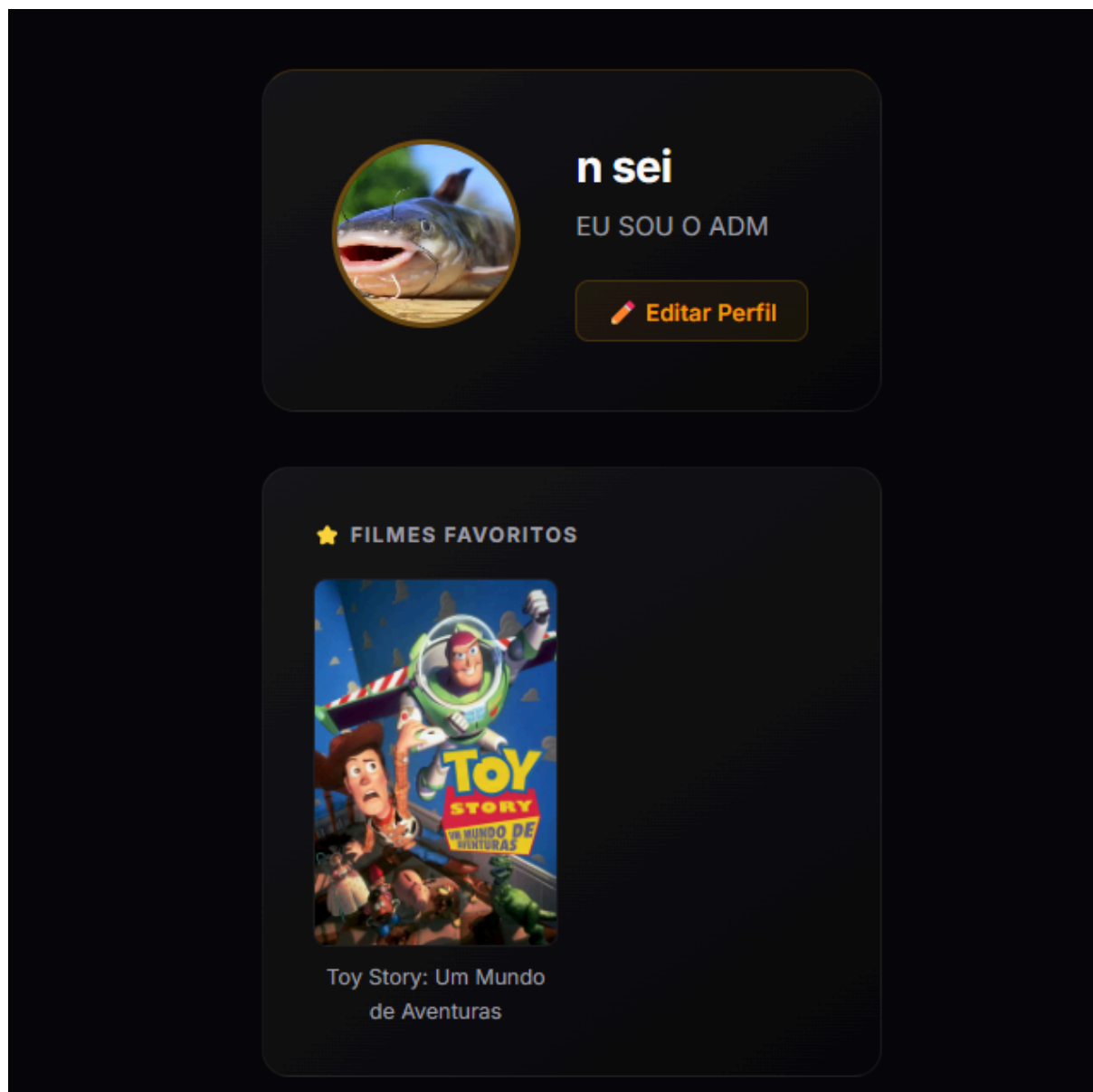


Figura 3: Atividade 6 – resultado (perfil do usuário exibindo foto de avatar carregada do Garage S3 e bio)

Dificuldades e como foram resolvidas. Em outubro de 2025, a imagem oficial do MinIO foi descontinuada do Docker Hub, motivando a migração definitiva para o Garage S3. Além disso, URLs pré-assinadas geravam links com o host interno do contêiner (garage:3900), inacessível aos navegadores dos clientes. A substituição por streaming direto no backend resolveu a compatibilidade universalmente em ambientes locais e Cloudflare.

Considerações finais

O percurso de desenvolvimento percorrido ao longo do primeiro bimestre de ISW055 permitiu consolidar na prática os fundamentos de arquiteturas em nuvem modernas. A transição de um script inicial monolítico em Python para um ecossistema completo composto por microsserviços containerizados em Node.js, mensageria com Redis e storage compatível com S3 proporcionou um aprendizado expressivo sobre segregação de responsabilidades e resiliência.

A maior complexidade encontrada ocorreu na Atividade 6, onde a descontinuidade do MinIO no Docker Hub exigiu adaptabilidade imediata para selecionar, configurar e implantar o Garage S3 em Rust, ajustando o layout de cluster e o roteamento interno de rede. Essa experiência evidenciou o valor de adotar padrões abertos de mercado (como a S3 API), permitindo trocar a engine de armazenamento sem alterar as rotas da aplicação.

Como aprendizado contínuo, destaca-se a relevância de conceber sistemas com foco em segurança desde a camada de design (Security by Design). A auditoria baseada no OWASP Top 10 permitiu antecipar vetores de ataque como enumeração de credenciais e envenenamento de links. Para o segundo bimestre, pretende-se aprofundar na instrumentação de métricas com Prometheus e automação de pipelines de entrega contínua (CI/CD).

Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Lucas Tetsuya Hiratuca

Pompeia, 07/10/2026